Combining tables Activity Book

Data Modeling for Qlik Sense

© 2023 QlikTech International AB. All rights reserved. All company and/or product names may be trade names, trademarks and/or registered trademarks of the respective owners with which they are associated.

DMQS May 2023

Table of Contents

Combining tables Activity Book

Table of Contents

Activity 1: Map DivisionName to Customers

Challenge 3
Hints 3
Solution 4

Activity 2 - Challenge: Map ShipperName to Orders

Challenge 5
Hints 5
Solution 6

Activity 3: Show preceding Load on preceding Load

Challenge 7
Hints 7
Solution 9

Activity 4: Join Categories with Products

Challenge 9
Hints 10
Solution 12

Activity 5: Join OrderDetails and Shipments

Challenge 12
Hints 13
Solution 14

Activity 6 - Challenge: Join more dimension tables

Challenge 15
Hints 15
Solution 16

Activity 1: Map DivisionName to Customers

Summary:

- Re-engineer the Divisions table as a mapping table.
- Apply the Divisions mapping table to the Customers table to fold the Divisions dimension into Customers.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, apply a **MAPPING LOAD** to the *Divisions* table. Move the *DivisionName* field into the *Customers* table by doing the following:

- Create a new script section after **Main** and name it *MappingTables*.
- Cut the *Divisions* **LOAD** statement and paste it into the **MappingTables** section.
- Change the name to *Divisions_Map* and add the **MAPPING** prefix in front of the **LOAD** statement.
- Use the **ApplyMap** function in the *Customers* table and comment out the *DivisionID* field, as it is no longer needed.
- Load your data and verify that the *Divisions_Map* does not exist in the data model and the *DivisionName* field has been added to the *Customers* table.

Hints

- The **MAPPING LOAD** technique eliminates the need for a separate *Divisions* table and the *DivisionID* field. It reduces the number of tables in your data model by one and denormalizes your schema.
- It is always recommended to verify that the values in the new field, added with the **ApplyMap** function, are correct.

Step-by-step guide

1. Create a new section immediately after **Main**. Name it *MappingTables*.
2. Position the cursor at the top of the **MappingTables** section and use the **Insert Connection String** option on the **ABC** data connection, to insert a **LIB CONNECT** statement.
3. From the **Customers** section, cut the entire **LOAD** statement for the *Divisions* table and paste it beneath the **LIB CONNECT** on the **MappingTables** section.
4. In the **MappingTables** section, change the table name from *[Divisions]:* to *[Divisions_Map]:*
5. Add the prefix **MAPPING** in front of the **LOAD** statement in the *Divisions_Map* table. The new script resembles the following:

```
// **** Load Divisions table ****
[Divisions_Map]:
MAPPING LOAD
    DivisionID,
    DivisionName;
SELECT DivisionID,
```

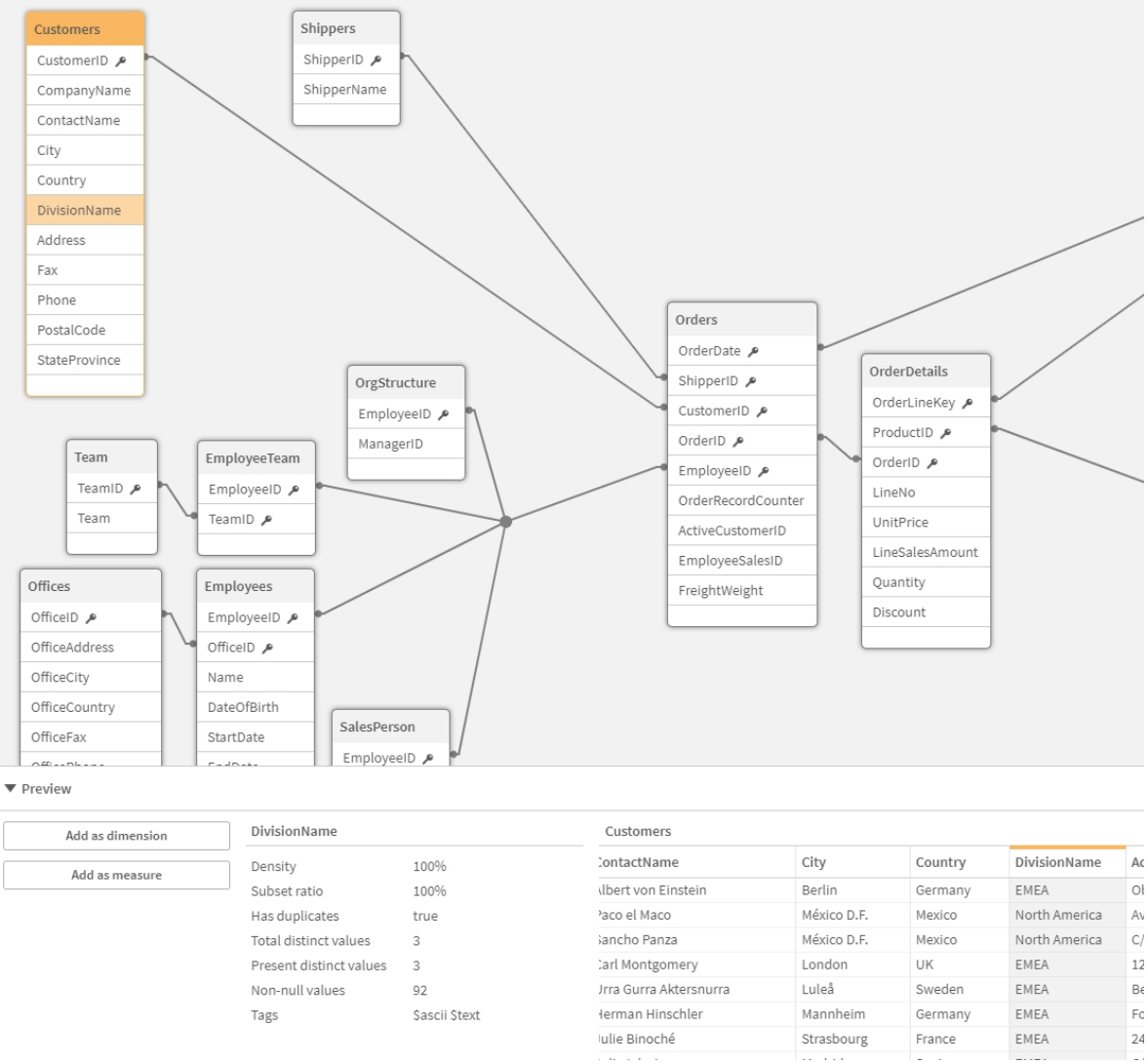
```
   DivisionName
FROM "ABC_2025".dbo.divisions;
```

6. In the **Customers** section, add this script to the *Customers* preceding **LOAD** statement, beneath the *DivisionID* line, to generate a new field within the table:

```
APPLYMAP('Divisions_Map', DivisionID) AS DivisionName,
```

7. Comment out *DivisionID* in the *Customers* table, as it is no longer needed.
8. Load your data and observe the differences in your data model: There is no longer a *Divisions* table, the *DivisionID* is gone, and the *DivisionName* field now exists in the *Customers* table.
9. Preview the data for *Customers* to check that *DivisionName* has been mapped correctly.

Solution



Activity 2 - Challenge: Map ShipperName to Orders

Summary:

- Re-engineer the Shippers dimension to act as a mapping table.
- Use an ApplyMap() function in the Orders table to incorporate ShipperName into Orders.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, use a **MAPPING LOAD** to move the *ShippersName* field into the *Orders* table by completing the following tasks:

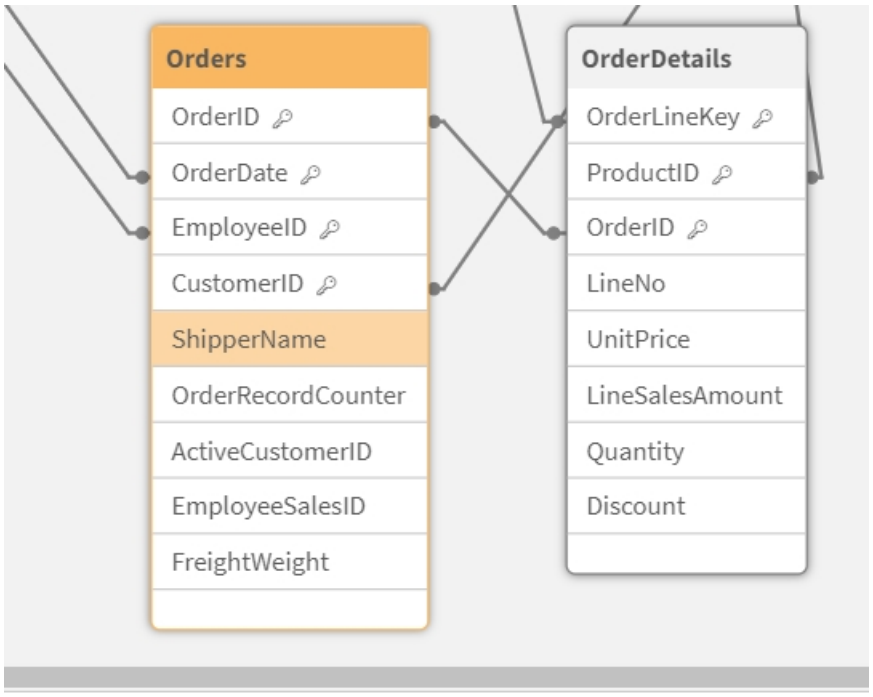
- Move the *Shippers* **LOAD** statement from the **Shipments** script section to the **MappingTables** script section. Relabel the **LOAD** statement *Shippers_Map*.
- Convert the table into a mapping table and remove the alias from the *CompanyName* field, as it is not needed.
- The *ShipperID* is the key field and must always be first in the **LOAD** statement. Swap the order of the fields if you need to.
- In the *Orders* table, use the **ApplyMap** statement to import the *CompanyName* field from the *Shippers_Map* table, based on matching *ShipperID* values.
- In the **ApplyMap** statement, add the word *MISSING* as the default value in case there are no matching values with the *Shippers_Map* table and name the new field *ShipperName*.
- Comment out the *ShipperID* field from the *Orders* table, as it is no longer needed in the data model.
- Load your data and verify that the values loaded in the *ShipperName* field are correct.
- Open the **Calendar Flags** app sheet and add a **Filter pane** with the *ShipperName* dimension. Verify that you see the value *MISSING* in the field.

Note: The objective of this **challenge** is to test whether you feel comfortable with the concepts that have been discussed so far in your class. There is no **Step-by-Step** section in this activity, so you must only rely on the **Hints** below to understand the tasks you have to execute.
As you are still new to many of these concepts, please don't hesitate to reach out for more guidance (help file, instructor, Qlik Community).

Hints

- This process eliminates the need for a separate *Shippers* table and the *ShipperID* field. It reduces the number of tables in your data model by one and denormalizes your schema.

Solution



Orders

OrderID	OrderDate	EmployeeID	CustomerID	ShipperName
3217	6/28/2023	16	4	SafeAndFast
1154	6/29/2022	13	79	SafeAndFast
3244	7/2/2023	45	34	SafeAndFast
2006	12/2/2022	33	1	SafeAndFast
2694	4/3/2023	19	76	Quik Shipping
3252	7/4/2023	45	34	SafeAndFast
671	1/6/2022	13	19	SafeAndFast
3094	6/6/2023	19	68	Quik Shipping

Activity 3: Show preceding Load on preceding Load

Summary:

- Create a new mapping table to hold ProductID and UnitCost.
- Apply the mapping table to OrderDetails and create an expression to calculate the cost of goods sold.
- Leverage an additional preceding LOAD on OrderDetails that uses the cost of goods sold and calculates profit margin.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, create a new field in the *OrderDetails* table to show the profit margin for every order line. Achieve this by completing the following:

- In the **MappingTables** script section, create a new mapping table, *UnitCost_Map*. Use the *Products* table, and only load *ProductID* and *UnitCost* fields from the original data source.
- In the *OrderDetails* table, use the **ApplyMap** function to create the *CostOfGoodsSold* field. The field is a calculated field and uses the following expression:
  `CostOfGoodsSold = UnitCost * Quantity`
- Add a preceding **LOAD** on preceding **LOAD** to create the *Margin* field by using the following calculation:
  `Margin = LineSalesAmount - CostOfGoodsSold`
- Load your data and check your data model.
- Create a new app sheet named *Margin* and add a **Bar chart** with *CategoryName* as the dimension and *Margin* as the measure, using the *Sum(Margin)* as the expression.
- Add two more measures, *Sum(LineSalesAmount)* as *Sales* and *Sum(CostOfGoodsSold)* as *COGS*.

Hints

- The original **ApplyMap** function needs to be used instead of *UnitCost* to create the *CostOfGoodsSold*.

Step-by-step guide

1. At the end of the **MappingTables** script section, add a couple carriage returns.
2. Use the data connection you previously used for the *Products* table, to load the following two fields:
   - *ProductID*
   - *UnitCost*

   ProductID is the key field. Make sure it is loaded first.
3. Remove the additional **LIB CONNECT** line which was added.
4. Remove the *[Products]:* table label.
5. Add the **MAPPING** prefix in the **LOAD** statement and label the new table *UnitCost_Map*.
6. In the **Orders** section, locate the *OrderDetails* table and add the following line to the preceding **LOAD**:

   ```
   APPLYMAP('UnitCost_Map', ProductID, NULL()) * Quantity AS CostOfGoodsSold,
   ```

   Copy to clipboard

   This expression calculates *CostOfGoodsSold* by mapping the *UnitCost* value from the *UnitCost_Map* table, and multiplies the value by the *Quantity*. If the value is not found in the mapping table, the result is *Null*.

7. Add a calculation for *Margin* based on the values for *LineSalesAmount* and *CostOfGoodsSold*. Note, you cannot use these fields in the preceding **LOAD** because they have not yet been created. You can, however, stack a second preceding **LOAD** on top of THIS preceding **LOAD** to use the fields there. This second preceding **LOAD** is executed after the execution of the first preceding **LOAD**.
   Add the following script before the **LOAD** in the *OrderDetails* table:

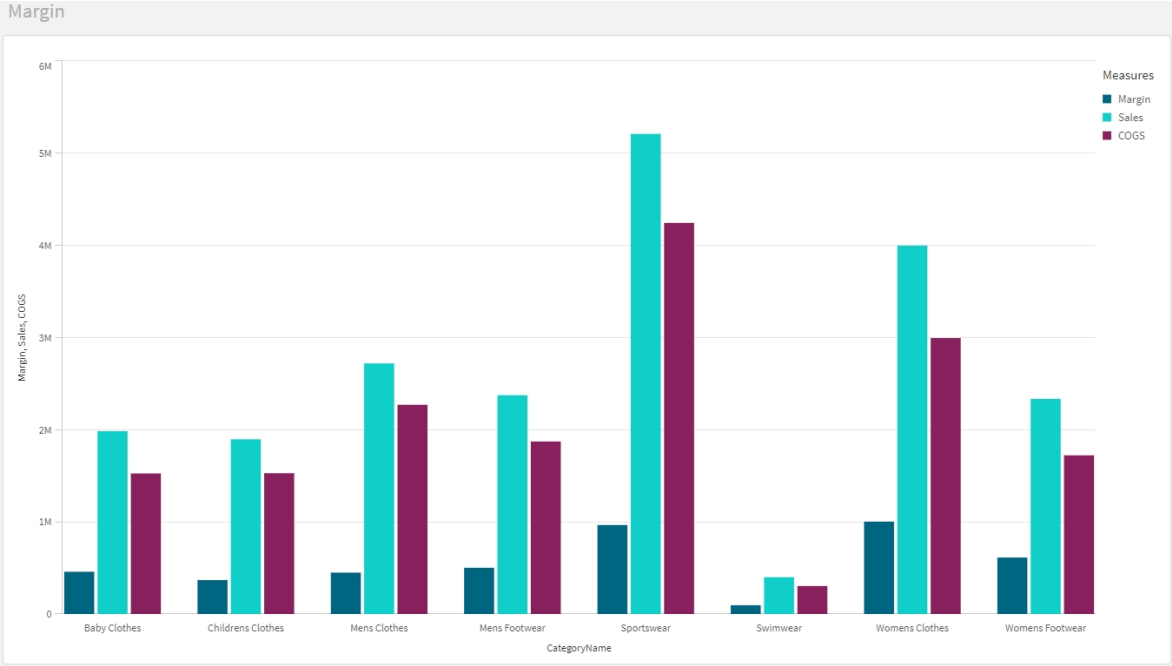
```
LOAD
    *,
    LineSalesAmount - CostOfGoodsSold AS Margin;
```

   The *OrderDetails* table now resembles the following syntax. However, the order of the fields within the preceding load statements may differ:

```
// **** Load OrderDetails from table ****
OrderDetails:
LOAD
    *,
    LineSalesAmount - CostOfGoodsSold AS Margin;
LOAD
    OrderID,
    LineNo,
    AUTONUMBER(OrderID & '|' & LineNo, 'OrderLine') AS OrderLineKey,
    ProductID,
    APPLYMAP('UnitCost_Map', ProductID, NULL()) * Quantity AS
CostOfGoodsSold,
    UnitPrice,
    UnitPrice * Quantity * (1 - Discount) AS LineSalesAmount,
    Quantity,
    Discount
WHERE EXISTS(OrderID, OrderID);
SELECT OrderID,
    "LineNo",
    ...
```

8. Load your data and view the data model. Preview the *OrderDetails* table data to examine the values for *Margin*.
9. Within **Analyze**, create a new app sheet, *Margin*.
10. Add a **Bar chart** to the new sheet, with *CategoryName* as the dimension and *Sum(Margin)* as the measure. Name the measure *Margin*.
11. Add the following two measures to the bar chart:
    - *Sum(LineSalesAmount)* with *Sales* as label.
    - *Sum(CostOfGoodsSold)* with *COGS* as label.
12. Resize the visualization to fill the sheet and exit edit mode to analyze the results. Make sure the chart resembles the solution image.

Solution



Activity 4: Join Categories with Products

Summary:

- Evaluate the association between the Categories and Products tables.
- Join the tables to further fold the data model in from a snowflake to a star schema.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, join the two tables, *Categories* and *Products*, by completing the tasks below:

- In your data model, analyze the *Categories* and *Products* tables and review the following:
  - Number of rows or records
  - Number of fields or columns
  - Field(s) linking the tables
  - **Density** of the *CategoryID* key field
  - **Subset ratio** of the *CategoryID* key field
  - Tables linked with other tables
  - Key fields in each table

    - Field data types: Are there any numeric fields that can be affected in case a duplication happens?
- Create a new app sheet, *Categories and Products,* and add two tables:
    - The first table contains two columns: *ProductID* as the dimension and *Count(CategoryID)* as the measure.
    - The second table contains *CategoryID* as the dimension and *Count(ProductID)* as the measure.
- In the **Products** script section, locate the *Categories* table.
    - Comment out the name of the table.
    - Add the **JOIN** prefix in front of the **LOAD** statement. Make sure that you specify the table you want to join. In this case, choose *Products* by adding it inside the brackets.
- Load your data and check the results in both the **Categories and Products** sheet and your data model.

Hints

- Joining tables is a technique that contributes to the denormalization of the data model.
- The type of **JOIN** needs to be defined first, before being applied in the script.

Step-by-step guide

1. Open the **Data model viewer** for the **ABC Sales Data** app.
2. Analyze the *Categories* and *Products* tables by reviewing the following:
    - Number of rows or records. (8 in *Categories* and 78 in *Products*)
    - Number of fields or columns. (3 in *Categories* and 11 in *Products*)
    - Which field or fields are linking the tables? *(CategoryID)*
    - **Density** of the *CategoryID* key field. (100% both)
    - **Subset ratio** of the *CategoryID* key field. (100% both)
    - Are any of these tables linked with other tables? (Yes, *Products* is linked with other tables)
    - What are the key fields in each table? *(CategoryID*, *SupplierID, ProductID)*
    - Preview the data and review the data types of the fields: Are there any numeric fields that can be affected in case a duplication happens? (Yes, in *Products*: *ItemCost, ItemPrice, UnitCost, QuantityperUnit, CataloguePrice, UnitsInStock, UnitsOnOrder)*

    Note: In a 1 to 1 relationship, one record from one of the tables corresponds to only one record of the other table. The *Categories* table has fewer records than the *Products* table. In this case, the relationship is one record corresponding to several records (1 to N).

3. Create a new app sheet to analyze the table relationship further. Name the sheet *Categories and Products*.
4. Insert a **Table** onto the left half of the new sheet, with *ProductID* as the dimension and *Count (CategoryID)* as the measure.
5. Insert a second **Table** onto the right half of the sheet, with *CategoryID* as the dimension and *Count(ProductID)* as the measure.

6. Make sure both tables are sorted by measure.
7. Finish editing the sheet and explain the relationship between *Categories* and *Products* (1 to N). Consider the technique (**JOIN** or **MAPPING**) that works best.
   The relationship between *Categories* and *Products* is 1 to N. There is no risk of duplicating the records in the *Products* table if we combine both tables using a **JOIN**. **JOIN** is best in this case because:
   - A **JOIN** will produce a table with 78 records and 13 fields, with no *NULL* values.
   - A **Mapping Table** only accepts two fields and the *Categories* table has three fields. If you want to use this technique, you need to create and maintain two mapping tables.
8. In the **Data load editor**, locate the **Products** section.
9. The *Categories* table contains a field called *Description*. After the **JOIN**, this field will be part of the *Products* table. To avoid confusion, it is recommended that you alias the field to *CategoryDescription.* This specifies that the *Description* field refers to *Categories*.

```
...
Description AS CategoryDescription
...
```

10. Comment out the *Categories* table name and use the **JOIN** prefix. Note, if the type of **JOIN** is *not* specified, **Outer Join** will be the result:

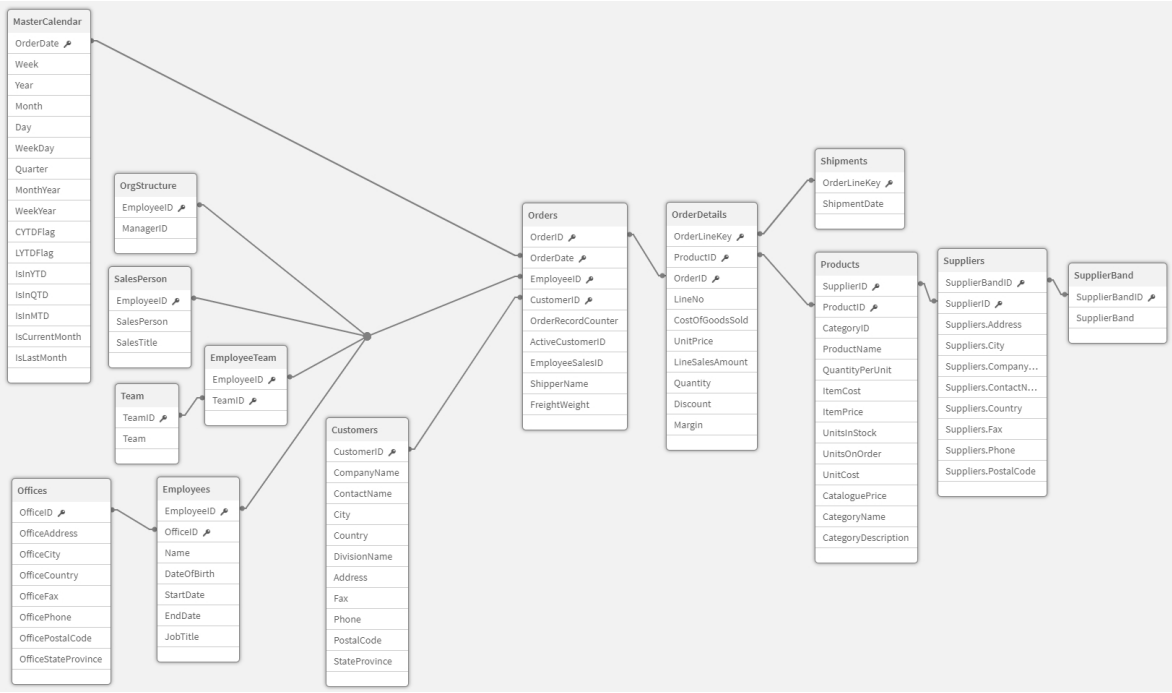
```
//Categories:
JOIN(Products)

LOAD CategoryID,
...
```

11. Load your data. Check the results in the **Categories and Products** sheet and in the **Data model viewer**. Notice that the fields of the *Categories* table have now been added to the *Products* table.
    The number of tables in your data model has been reduced by one. You are one step closer to a star schema.

> **Note:** The above steps explain in detail why it is acceptable to use **Join** to form two tables into one. If you do not specify the **Join** type, the **Join** prefix will combine the two tables using **Outer Join**.
> It is a best practice to explicitly define the **Join** type you want to use. In this case, although using **Join** will transform the data model in the same way as a **Left Join**, it is recommended to use **Left Join**.

Solution



Activity 5: Join OrderDetails and Shipments

Summary:

- Analyze the association between OrderDetails and Shipments.
- Use the correct JOIN type between the tables to fold Shipments into OrderDetails.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, combine the *OrderDetails* and *Shipments* tables by completing the following:

- Analyze the *OrderDetails* and *Shipments* tables. Review the number of **Rows**, **Density** and **Subset ratio** for the key fields.
- Create a new app sheet, *Shipments,* and insert two tables onto the new sheet.
  - The first table contains four dimension columns: *OrderLineKey*, *OrderID*, *LineNo* and *ShipmentDate*.
  - The second table contains three dimension columns: *OrderLineKey*, *OrderID,* and *LineNo,* and *Count(Distinct ShipmentDate)* as the measure.
- In the **Shipments** script section, locate the *Shipments* table. Comment out the name of the table and add the **JOIN** prefix above the **LOAD** statement: *Left Join (OrderDetails).*

- Comment out the **WHERE** clause from the *Shipments* table, as it is no longer needed.
- Load your data. Check the results in the new sheet and in your data model.

Hints

- The *Shipments* table only contains a date field (*ShipmentDate*) and the key field (*OrderLineKey*). In this case, *OrderDetails* is the main table, as it is one of the fact tables in the data model. You can keep *OrderDetails* as the main table and include the information from the *Shipments* table in it.

Step-by-step guide

1. In your data model, analyze the table information for *OrderDetails* and *Shipments*. Similar to the previous activity, review the number of **Rows**, **Fields**, information **Density**, and **Subset ratio** of the key field.
2. Create a new app sheet named *Shipments* and insert a new **Table** onto it. Expand the **Table** to cover the top half of the sheet.
3. Add the field *OrderLineKey* as the first dimension. Use the **Properties** panel to add three more dimensions into your table: *OrderID*, *LineNo,* and *ShipmentDate*.
4. Insert a second **Table** onto the bottom half of the sheet.
5. Add the field *OrderLineKey* as the first dimension. Add two more dimensions: *OrderID* and *LineNo*.
6. Use the **Properties** panel again to add the *Count(ShipmentDate)* as the measure.
7. Modify the measure expression and add the *Distinct* qualifier before the *ShipmentDate* field as below:
   `Count(Distinct ShipmentDate)`
8. Sort the table by the *Count(Distinct ShipmentDate)* measure.
9. Disable the edit mode. Notice that there is only one *ShipmentDate* maximum per each *OrderLineKey* based on the count above. You also see that not all the *OrderLineKey* values have a *ShipmentDate*. Scroll down to verify that.
10. There is no risk of duplicating *OrderDetails* records if you join the *Shipments* table to them. In the **Data load editor**, navigate to the **Shipments** script section and locate the *Shipments* table. Note that this table is loaded after the *OrderDetails* table. The order of the tables is relevant for the type of **JOIN** you use for the **LEFT** and **RIGHT JOIN**.
11. What type of **JOIN** are you going to apply? In this case, the main table is *OrderDetails* and you only want to add the *ShipmentDate* field to it. Analyze the results using the different types of **JOIN**:
    - **INNER JOIN** removes all the *OrderLineKey* values with no *ShipmentDate*, and only leaves the common *OrderLineKey* values in both tables. This is not the desired result.
    - **RIGHT JOIN** uses the second table (*Shipments* table) as the main table, and removes those *OrderLineKey* values with no shipment dates. This is not the desired result.
    - **LEFT JOIN** uses the first table (*OrderDetails* table) as the main table, and removes those *OrderLineKey* values in the *Shipments* table that do not exist in the *OrderDetails* table. This is what you need.
    - **OUTER JOIN/JOIN** keeps all the records from both tables. This type of **JOIN** works, but the **WHERE** clause in *Shipments* is required.

    Comment out the **[Shipments]:** table label and add a **LEFT JOIN** in front of the **LOAD**

statement for this table.

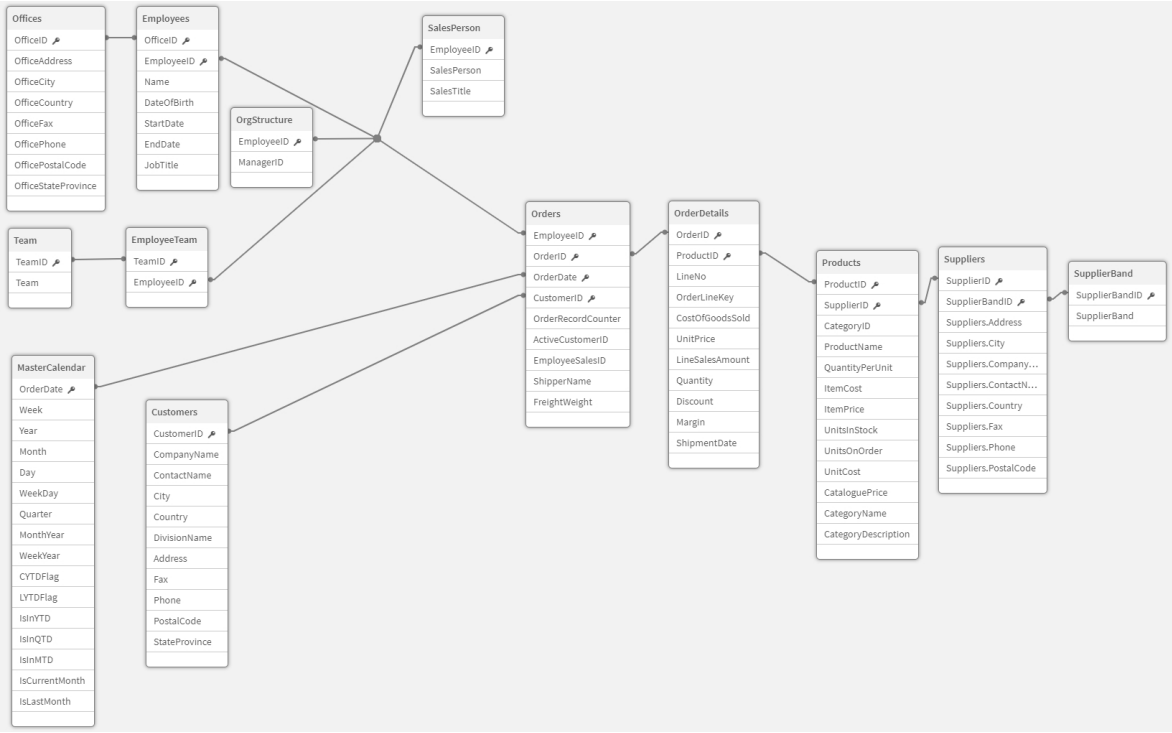
```
//Shipments:
LEFT JOIN (OrderDetails)
LOAD
...
```

12. The **WHERE** clause in the *Shipments* table is no longer required due to the nature of the **LEFT JOIN**. Comment out the **WHERE** clause in the *Shipments* table. Make sure you have a semi-colon at the end of the statement.

```
...
//WHERE EXISTS(OrderID, OrderID)
;
```

13. Load the data and observe the changes in the data model. One more table, *Shipments,* has been removed and its fields have been added to *OrderDetails*.

Solution



Activity 6 - Challenge: Join more dimension tables

Summary:

- Continue folding outer dimensions into inner dimensions, by joining several more tables.
- Observe the changes in the data model by using the Data model viewer.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, combine more dimension tables by completing the following tasks:

- Join more tables together to create a data model that looks more like a star schema:
  - **LEFT JOIN** the *EmployeeTeam*, *Team*, *Offices* and *OrgStructure* (optional) tables to *Employees*. Comment out the name of each table you join.
  - **LEFT JOIN** the *Suppliers* and *SuppliersBand* tables to *Products*.

    Note: Because the **Qualify** statement needs the table name to add it as a prefix, keep the *Suppliers* table name uncommented.

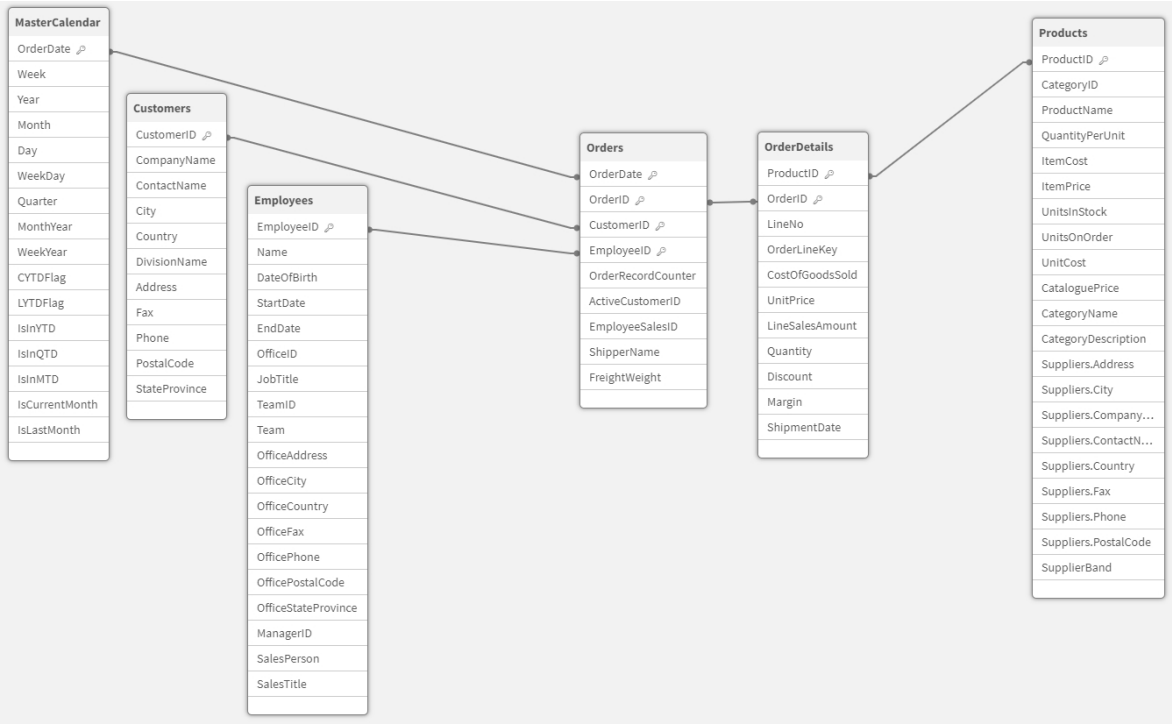
  - **LEFT JOIN** the *SalesPerson* table to *Employees*.
- Create a new script section at the end, named *LoadComplete*. Then, use a statement to drop the fields *SupplierID* and *SupplierBandID*, as they are no longer needed.
- Load the data and check your data model to identify the changes.

Note: The objective of this **challenge** is to test whether you feel comfortable with the concepts that have been discussed so far in your class. There is no **Step-by-Step** section in this activity, so you must only rely on the **Hints** below to understand the tasks you have to execute.
As you are still new to many of these concepts, please don't hesitate to reach out for more guidance (help file, instructor, Qlik Community).

Hints

- Joining more dimension tables optimizes the performance of the Qlik Sense app and brings the data model a step closer to a star schema.

Solution


MasterCalendar
OrderDate
Week
Year
Month
Day
WeekDay
Quarter
MonthYear
WeekYear
CYTDFlag
LYTDFlag
IsInYTD
IsInQTD
IsInMTD
IsCurrentMonth
IsLastMonth
Customers
CustomerID
CompanyName
ContactName
City
Country
DivisionName
Address
Fax
Phone
PostalCode
StateProvince
Employees
EmployeeID
Name
DateOfBirth
StartDate
EndDate
OfficeID
JobTitle
TeamID
Team
OfficeAddress
OfficeCity
OfficeCountry
OfficeFax
OfficePhone
OfficePostalCode
OfficeStateProvince
ManagerID
SalesPerson
SalesTitle
Orders
OrderDate
OrderID
CustomerID
EmployeeID
OrderRecordCounter
ActiveCustomerID
EmployeeSalesID
ShipperName
FreightWeight
OrderDetails
ProductID
OrderID
LineNo
OrderLineKey
CostOfGoodsSold
UnitPrice
LineSalesAmount
Quantity
Discount
Margin
ShipmentDate
Products
ProductID
CategoryID
ProductName
QuantityPerUnit
ItemCost
ItemPrice
UnitsInStock
UnitsOnOrder
UnitCost
CataloguePrice
CategoryName
CategoryDescription
Suppliers.Address
Suppliers.City
Suppliers.Company...
Suppliers.ContactN...
Suppliers.Country
Suppliers.Fax
Suppliers.Phone
Suppliers.PostalCode
SupplierBand